

BearLibTerminal / API / **Input**

functions

Input in the BearLibTerminal is handled with [read](#), [read_str](#), [has_input](#), [state](#) and [check](#) functions.

read

This function returns the next event from input queue:

```
int key = terminal_read();
```

```
if (key == TK_ESCAPE)
{
    // Quit
}
```

If there is no more events in the queue, **read** will wait for an event to come. If this blocking behaviour is not desirable, you can check [has_input](#) before calling **read**.

peek

Similar to [read](#) but does not remove the event from the internal queue. The next **peek** or **read** call will return the same value. This is non-blocking function: if the queue is empty, it will not wait and return 0.

read_str

This function does a simple blocking read of a string without any parsing. User input is displayed at specified coordinates and also limited to specified length so it can be used in game interface. The function displays user input within current layer and restores the scene before return. It is an application's responsibility to keep the result on screen.

The function returns the size of a string if user has confirmed input by pressing Enter, or TK_INPUT_CANCELLED if operation was cancelled by pressing ESCAPE or closing the window.

```
wchar_t s[32] = L"Hello";

if (terminal_read_wstr(2, 1, s, sizeof(s)-1) >= 0)
{
    terminal_wprintf(L"You entered: \"%ls\"", s);
}
```

If **read_str** does not fit your needs, see [TK CHAR/TK WCHAR](#) states allowing for manual text input.

has_input

This function returns a boolean value indicating immediate input availability. If **has_input** returns *true*, the next call to [read](#) is guaranteed not to block.

```
while (terminal_has_input())
```

```
{
    int key = terminal_read();
}
```

state

This function queries the current numeric value of a library property. For input it is used to get different event properties, see the [table](#) below.

```
int key = terminal_read();

if (key == TK_MOUSE_MOVE)
{
    int x = terminal_state(TK_MOUSE_X);
    int y = terminal_state(TK_MOUSE_Y);
}
```

Note that values returned by **state** function may not reflect the exact real-time value of a property, but the “current” one in respect to the input queue and already dequeued events. [More...](#)

check

This is a simple wrapper around [state](#) function. In a lot of programming languages numerical and boolean types are not compatible so checking logically boolean properties with [state](#) may become unnecessary verbose. So you can use **check** to test some property.

```
int key = terminal_read();

if (key == TK_Z)
{
    if (terminal_check(TK_SHIFT))
    {
        // 'Z': zap a wand
    }
    else
    {
        // 'z': cast a spell
    }
}
```

Input options

These options are configured by the [set](#) function:

```
terminal_set
(
    "input:"
    "cursor-symbol = 0x1F,"
    "cursor-blink-rate = 500,"
    "precise-mouse = false,"
```

```
"mouse-cursor = true,"
"filter=[keyboard];"
);
```

input.cursor-symbol

Character/tile used as a cursor by [read_str](#) function. Default is “_” (underscore) and can be changed to match application design, e. g. to full square or vertical bar.

input.cursor-blink-rate

Speed at which [read_str](#)'s cursor blinks, in milliseconds. Default is 500, i. e. twice a second.

input.precise-mouse

A boolean flag controlling how mouse movement events are generated. When false, an event is generated only when cursor is moved from one cell to another. When true, any mouse movement generates an event. Default is false.

input.mouse-cursor

A boolean flag controlling mouse cursor visibility. Default is true.

input.filter

It is a list of events the application is interested in. All other will be silently processed by the library, thus [read](#) will return only events from this list:

```
input.filter = [keyboard, mouse+]
```

A name in the list may specify:

- an event group: 'keyboard', 'mouse', 'keypad', 'arrows'.
- a single event (name as in the table below, but without 'TK_' prefix; case-insensitive).
- a set of alphanumeric keys, e. g. 'wasd' or '0123456789'; obviously it must differ from event group names.

Mentioning the events enables reading them. If there is a '+' (plus sign) added to the name, both keypresses and keyreleases will be read.

Default filter is 'keyboard'. To enable the mouse support, set the filter to 'keyboard, mouse'. System events like TK_CLOSE and TK_RESIZED are always enabled.

There is no input filtering if the list is empty. Therefore filtering is disabled by assigning 'none' or an empty list to the *input.filter* option.

Event and state constants

Group	Constant	Description
Alphanumeric	TK_A ... TK_Z TK_0 ... TK_9 TK_SPACE	Event: indicates that corresponding key was pressed or released. If key was released, its code is combined with TK_KEY_RELEASED flag (more...).

	TK_MINUS	-	
	TK_EQUALS	=	State: current state of the key (see input queue).
	TK_LBRACKET	[	
	TK_RBRACKET	]	
	TK_BACKSLASH	\	
	TK_SEMICOLON	;	
	TK_APOSTROPHE	'	
	TK_GRAVE	`	
	TK_COMMA	,	
	TK_PERIOD	.	
	TK_SLASH	/	
	TK_F1 ... TK_F12		
	TK_RETURN		
	TK_ESCAPE		
	TK_BACKSPACE		
	TK_TAB		
	TK_PAUSE		
	TK_INSERT		
	TK_HOME		
Functional, navigation, modifiers	TK_PAGEUP		
	TK_DELETE		
	TK_END		
	TK_PAGEDOWN		
	TK_RIGHT		
	TK_LEFT		
	TK_DOWN		
	TK_UP		
	TK_SHIFT		
	TK_CONTROL		
	TK_KP_0 ... TK_KP_9		
	TK_KP_DIVIDE	/	
	TK_KP_MULTIPLY	*	
	TK_KP_MINUS	-	
	TK_KP_PLUS	+	
Keypad	TK_KP_PERIOD	.	
	TK_KP_ENTER		
Mouse	TK_MOUSE_LEFT		
	TK_MOUSE_RIGHT		
	TK_MOUSE_MIDDLE		
	TK_MOUSE_X1		
	TK_MOUSE_X2		
	TK_MOUSE_MOVE		Event: generated when mouse coordinates are changed by at least one cell. If input.precise-mouse option is set, this event is generated for any mouse movement, even within single cell.

Properties	TK_MOUSE_SCROLL	Event: generated when mouse wheel is turned.
	TK_MOUSE_WHEEL	State: returns scroll delta of the last wheel event (more...).
	TK_MOUSE_X	State: current mouse coordinates in cells.
	TK_MOUSE_Y	
	TK_MOUSE_PIXEL_X	State: current mouse coordinates in pixels. These states are available regardless of input.precise-mouse option.
	TK_MOUSE_PIXEL_Y	
	TK_MOUSE_CLICKS	State: number of fast consecutive clicks for last mouse button event, i. e. 1 for regular click, 2 for double click, etc. (more...)
	TK_WIDTH	State: size of a terminal in cells.
	TK_HEIGHT	
	TK_CELL_WIDTH	State: size of a single cell in pixels.
	TK_CELL_HEIGHT	
	TK_COLOR	State: current foreground color.
	TK_BKCOLOR	State: current background color.
	TK_LAYER	State: current layer index.
	TK_COMPOSITION	State: current composition flag.
	TK_CHAR	State: unicode or translated ANSI character code produced by the most recent event (more...).
	TK_WCHAR	
	TK_EVENT	State: code of the last dequeued input event.
	TK_FULLSCREEN	State: fullscreen mode status.
Other events	TK_CLOSE	Event: quit request, generated when user is trying to close the window, e. g. by clicking on close button or pressing Alt+F4 (see input.sticky-close option).
	TK_RESIZED	Event: window has been resized by user (see window.resizeable option).

Input queue

All input events generated by the library are queued in the internal input queue. This queue is accessed via the [read](#) call and until the event is dequeued by the application, it will not have any effect on the input states listed above. This has a subtle but important consequence: all of the input states are always consistent.

Say the user presses A, then Shift+B. It produces several events: `A` `A` `Shift` `B` `Shift` `B`. When application processes input and encounters some keypress event, it can check whether it was a single key or a combination by a simple call to the [check](#) function:

```
if (key == TK_A && terminal_check(TK_SHIFT))
{
    // Do something
}
```

For `A` this condition will evaluate to false. For `B` the situation is different because the `Shift` event will already be dequeued by that time.

In many libraries this problem (checking keypresses for combinations with other keys) is solved by introducing an event structure containing the pressed key code and a number of status flags for Shift, Ctrl, mouse keys, etc. BearLibTerminal approaches this a bit differently, making all input states behave like a single structure:

```
if (key == TK_F && terminal_check(TK_SHIFT))
```

```

{
    // Shift+F: search instead of fire.
}
else if (key == TK_MOUSE_LEFT && terminal_check(TK_CONTROL))
{
    // Ctrl+LMB: command instead of select
}
else if (terminal_state(TK_CHAR) == '?' && terminal_state(TK_MOUSE_Y) < 20)
{
    // '?' while mouse cursor is higher than 20th row: whatever it is
}

```

Note that some of the terminal properties are not queued and are always up-to-date because they do not have much to do with input and will not make much sense otherwise. These properties are: `TK_WIDTH`, `TK_HEIGHT`, `TK_CELL_WIDTH`, `TK_CELL_HEIGHT`, `TK_COLOR`, `TK_BKCOLOR`, `TK_LAYER`, `TK_COMPOSITION`, `TK_FULLSCREEN`.

TK_KEY_RELEASED flag

This constant is used to distinguish between keypresses and keyreleases. For keypress or auto-repeat the [read](#) function will return an event with the corresponding key code, e. g. `TK_A` or `TK_BACKSPACE`. For keyrelease this code will be combined with `TK_KEY_RELEASED` flag. It allows for easy filtering of key events because presses and releases have different codes but still can be matched:

```

int key = terminal_read();

if (key == TK_A)
{
    // 'A' was pressed
    foo();
}
else if (key == TK_B | TK_KEY_RELEASED)
{
    // 'B' was released
    bar();
}
else if ((key & ~TK_KEY_RELEASED) == TK_C)
{
    // 'C' was either pressed or released
    baz();
}

```

Note that for auto-repeat no intermediate keyreleases are reported, i. e. there may be several keypresses before single keyrelease.

TK_CHAR/TK_WCHAR states

These states provide means for custom text input. They return a character produced by the most recently dequeued event. `TK_WCHAR` provides an Unicode codepoint of the character and `TK_CHAR` provides an [ANSI](#) code translated according to [terminal.encoding](#) option.

```

std::wstring my_read_str(int x, int y, int max)
{
    std::wstring s;

    while (true)
    {
        int key = terminal_read();

        if (key == TK_RETURN || key == TK_ESCAPE || key == TK_CLOSE)
        {
            // No distinction between confirmation and interruption
            break;
        }
        else if (key == TK_BACKSPACE && s.length() > 0)
        {
            // Remove one character
            s.resize(s.length()-1);
        }
        else if (terminal_check(TK_WCHAR) && s.length() < max)
        {
            // Append one character
            s += (wchar_t)terminal_state(TK_WCHAR);
        }

        // Visual feedback
        terminal_wprintf(x, y, L"%-*ls_", max, s.c_str());
    }

    return s;
}

```

TK_MOUSE_WHEEL state

This state returns the direction and amount of steps the mouse wheel was rotated for during the most recently dequeued TK_MOUSE_WHEEL event, e. g. 1 or -2:

```

int key = terminal_read();

if (key == TK_MOUSE_SCROLL)
{
    int amount = terminal_state(TK_MOUSE_WHEEL);

    if (mouseOverFoo)
    {
        fooValue += amount;
    }
    else if (mouseOverBar)
    {
        barValue += amount;
    }
}

```

```
}  
}
```

TK_MOUSE_CLICKS state

This state returns the number of fast consecutive clicks happened up to the most recently dequeued mouse button-press event:

```
int key = terminal_read();  
  
if (key == TK_MOUSE_LEFT)  
{  
    int clicks = terminal_state(TK_MOUSE_CLICKS);  
  
    if (clicks == 1)  
    {  
        // Select the object under mouse cursor  
    }  
    else if (clicks == 2)  
    {  
        // Execute some action with selected object  
    }  
}
```

[Show pagesource](#)[Old revisions](#)[Media Manager](#)[Login](#)[Sitemap](#)[Back to top](#)